

PYTHON DAY 4

Loops, Iteration, and Repeating Work

Total time: 3 to 4 hours | **Main goal:** Learn how to repeat work safely and predictably without copy-pasting code.

Today's Big Idea

Programs often need to repeat an action: process users, retry input, count values, generate reports, or iterate through data. Today you learn the two fundamental beginner loop styles in Python: **while** and **for**.

```
Data or condition
↓
Choose repetition strategy
↓
while → repeat while a condition is true
for → iterate over items
↓
Optional control: break / continue
↓
Result
```

1. Warm-up - 15 minutes

Without opening Day 3, write a small program that asks for a number and prints whether it is positive, negative, or zero. Then think: what if you wanted to repeat that program 100 times? This is the problem loops solve.

2. The while Loop - 35 minutes

Create `while_basics.py`:

```
count = 1

while count <= 5:
    print(count)
    count = count + 1
```

Before running it, answer: What changes on every iteration? What condition eventually becomes false? What would happen if count never changed?

Deliberately create an infinite loop, observe it briefly, then stop it with Ctrl+C:

```
count = 1
while count <= 5:
    print(count)
```

Research task: Define these three parts of a loop: initialization, condition, and update. Explain how each appears in the working example.

3. Build a Countdown - 20 minutes

Create `countdown.py`. Start from 10 and count down to 1. After that print 'Done!'. Do not copy a finished solution. Decide what the starting value, condition, and update must be.

Test edge cases: start from 1, 0, and a negative number.

4. User-Controlled while Loop - 30 minutes

Create `password_retry.py`. Keep asking for a password until the correct value is entered.

```
correct_password = "python123"
password = ""

while password != correct_password:
    password = input("Enter password: ")
```

```
print("Access granted")
```

Improve it later so the user gets a limited number of attempts. For now, understand why the loop eventually stops.

5. The for Loop - 35 minutes

A for loop is commonly used when you want to iterate over items. Start with a string:

```
for letter in "Python":  
    print(letter)
```

Then iterate over a sequence of numbers using range():

```
for number in range(5):  
    print(number)
```

Research experiment: compare these expressions and predict their outputs before running them:

```
range(5)  
range(1, 5)  
range(1, 10, 2)  
range(10, 0, -1)
```

Important question: Why does range(5) stop at 4? Write your own explanation, then verify it using the official documentation.

Official reading: <https://docs.python.org/3/library/functions.html#func-range>

6. for vs while - 25 minutes

Write two programs that print 1 through 10. Use while in the first and for with range() in the second.

Then answer: Which version is easier to read when the number of repetitions is already known? When would while be more natural?

Do not treat one as universally better. The choice depends on the problem.

7. break - 25 minutes

```
for number in range(1, 11):  
    if number == 5:  
        break  
    print(number)
```

Predict whether 5 will print. Then change the break condition and experiment.

Practical experiment: repeatedly ask for input and stop when the user types quit.

```
while True:  
    command = input("Enter command: ")  
    if command == "quit":  
        break  
    print("You entered:", command)
```

Research question: Why can while True be useful even though accidental infinite loops are dangerous?

8. continue - 20 minutes

```
for number in range(1, 11):  
    if number % 2 == 0:  
        continue  
    print(number)
```

Explain why only odd numbers print. Change the condition to skip numbers divisible by 3.

9. Mini Project - Number Analyzer

Create number_analyzer.py. Ask the user for 5 numbers, one at a time. For each number, print whether it is positive, negative, or zero. Count how many numbers fall into each category.

For today, you may use separate counters such as positive_count, negative_count, and zero_count. Collections will come later.

10. Nested Loops - 30 minutes

A loop can run inside another loop. Start small:

```
for row in range(1, 4):
    for column in range(1, 4):
        print(row, column)
```

Before running, estimate how many times the inner print executes. Then calculate it.

Build a multiplication table for numbers 1 through 5. Focus on understanding the relationship between the outer and inner loops rather than memorizing syntax.

11. Practical Project - Guessing Game

Create `guessing_game.py`. Hard-code a secret number. Repeatedly ask the user to guess until they get it correct.

Requirements:

- Use a while loop.
- Convert input to int.
- Tell the user whether the guess is too high or too low.
- Count attempts.
- Stop when the correct answer is guessed.
- Test invalid or unexpected input if you can. Exception handling will be studied in depth later.

12. Main Project - CLI ATM Simulator

Build `atm.py` with a starting balance. Repeatedly show a menu until the user chooses to exit.

```
1. Check balance
2. Deposit money
3. Withdraw money
4. Exit
```

Requirements:

- Use a while loop for the main menu.
- Use if/elif/else for menu choices.
- Use a variable to track balance.
- Reject invalid menu choices.
- Do not allow withdrawal greater than the balance.
- Use break to exit cleanly.
- Test at least 10 user paths.

Suggested test paths: check balance immediately, deposit, withdraw successfully, attempt excessive withdrawal, invalid menu choice, multiple operations, and exit.

13. Debugging Lab - 20 minutes

Investigate and fix each example:

```
count = 1
while count <= 5:
    print(count)
    count = count - 1
```

```
for number in range(1, 10):
    print(number)
```

```
for number in range(5):  
    if number == 3:  
        continue  
    print(number)
```

For each one, identify the issue before changing the code. Ask: Is the condition wrong? Is the update wrong? Is indentation wrong? Is some code unreachable?

14. Research Challenge - How Does Iteration Work?

Do not attempt to master Python's iterator protocol today. Research these terms at a high level: iterable, iterator, iteration, and range object.

Experiment:

```
numbers = range(5)
print(numbers)
print(type(numbers))

for item in numbers:
    print(item)
```

Write a short explanation of why range does not simply print exactly like a list of numbers.

Official references: <https://docs.python.org/3/library/stdtypes.html> and <https://docs.python.org/3/glossary.html>

15. Knowledge Check

- Q1. What are the three basic parts of a typical while loop?
- Q2. What creates an infinite loop?
- Q3. When is while often more suitable than for?
- Q4. What does range(5) produce when iterated?
- Q5. What is the difference between break and continue?
- Q6. What does while True mean?
- Q7. How many times does an inner loop run in a 3 by 4 nested-loop structure?
- Q8. Why must a loop variable usually change in a while loop?
- Q9. What is an iterable at a high level?
- Q10. What test cases would you use for the ATM simulator?

16. Day 4 Completion Checklist

- ☐ I can write and explain a while loop.
- ☐ I know how initialization, condition, and update work together.
- ☐ I can recognize an infinite loop.
- ☐ I can use Ctrl+C to stop a runaway terminal program.
- ☐ I can write a for loop.
- ☐ I understand basic range() behavior.
- ☐ I can choose between for and while for simple problems.
- ☐ I can use break and continue.
- ☐ I understand the basics of nested loops.
- ☐ I completed Number Analyzer.
- ☐ I completed Guessing Game.
- ☐ I completed the CLI ATM Simulator.
- ☐ I tested multiple paths instead of only the happy path.

Recommended Study Schedule

15 min warm-up | 35 min while | 20 min countdown | 30 min user-controlled loop | 35 min for/range | 25 min for vs while | 25 min break | 20 min continue | 35 min mini project | 30 min nested loops | 45 min guessing game | 60 min ATM project | 20 min debugging/review.

Official References

Python Tutorial: <https://docs.python.org/3/tutorial/>

range(): <https://docs.python.org/3/library/functions.html#func-range>

Built-in Types: <https://docs.python.org/3/library/stdtypes.html>

Python Glossary: <https://docs.python.org/3/glossary.html>

Day 5 Preview

Next, you will learn strings deeply: indexing, slicing, common string methods, immutability, formatting, f-strings, and practical text-processing projects.